In the context of Prometheus, alerting comprises two distinct parts:

- *Prometheus itself*—You define what an alert is. This effectively means you define conditions using PromQL, and Prometheus evaluates them periodically. Once a condition is met, Prometheus considers this to be the alert "firing" and sends it to the Alertmanager API.
- *Alertmanager API*—This provides an API that enables clients (typically Prometheus) to send in alerts and takes care of the notifications. For example, you could define a policy that sends a notification for certain types of alerts via email or implement a policy such as *during work hours, send the notification to Slack; after work, send a text message*. But that's not the only thing the Alertmanager does: it deduplicates alerts and allows you to group and suppress notifications to reduce alert fatigue and lessen alert storms.

Figure 7.2 shows how Prometheus and Alertmanager work together to handle alerts. As we discussed in chapter 4, the scrape targets (e.g., your instrumented applications or Prometheus exporters, such as the Node exporter) expose metrics in the Prometheus exposition format, which Prometheus scrapes (in other words, it calls the endpoints, which are by default expected to be at /metrics). If you have alerting rules defined in Prometheus, Prometheus continuously evaluates them and calls the Alertmanager in case the defined threshold is reached. The Alertmanager, in turn, processes alerts with the goal of sending out as few and concise notifications as possible. Via the configured receivers in the Alertmanager, it delivers a notification to one or more destinations, where the operator (a human) finally consumes the notification and acts upon it accordingly.

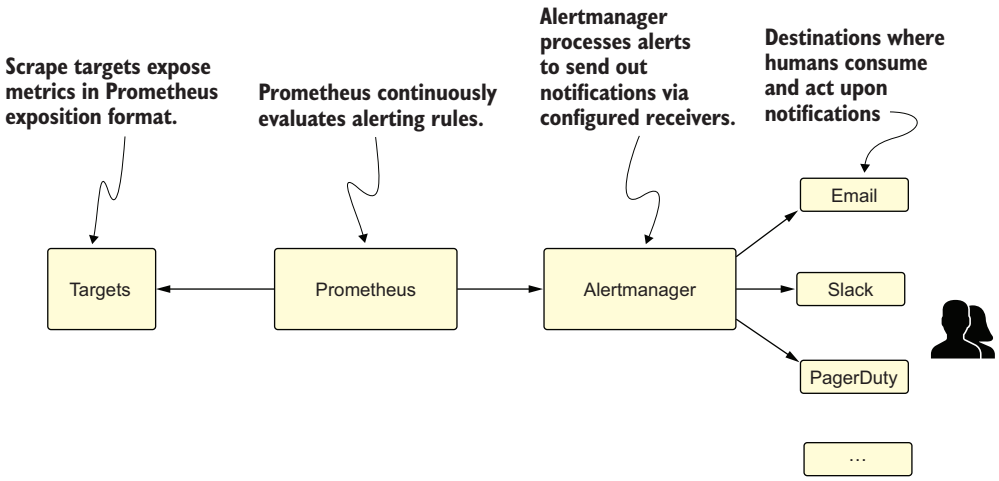


Figure 7.2 Prometheus and the Alertmanager